Programmazione Web e Mobile con Laboratorio

Lezione 9 - Basi di JavaScript 1 (Sintassi, variabili, funzioni)

JavaScript

- Linguaggio di programmazione usato per aggiungere interattività alle pagine web
- Caratteristiche principali:
  - funziona principalmente lato client (nel browser dell’utente)
  - modifica il contenuto HTML e CSS in tempo reale
  - esegue azioni in risposta ad eventi (click, input, …)



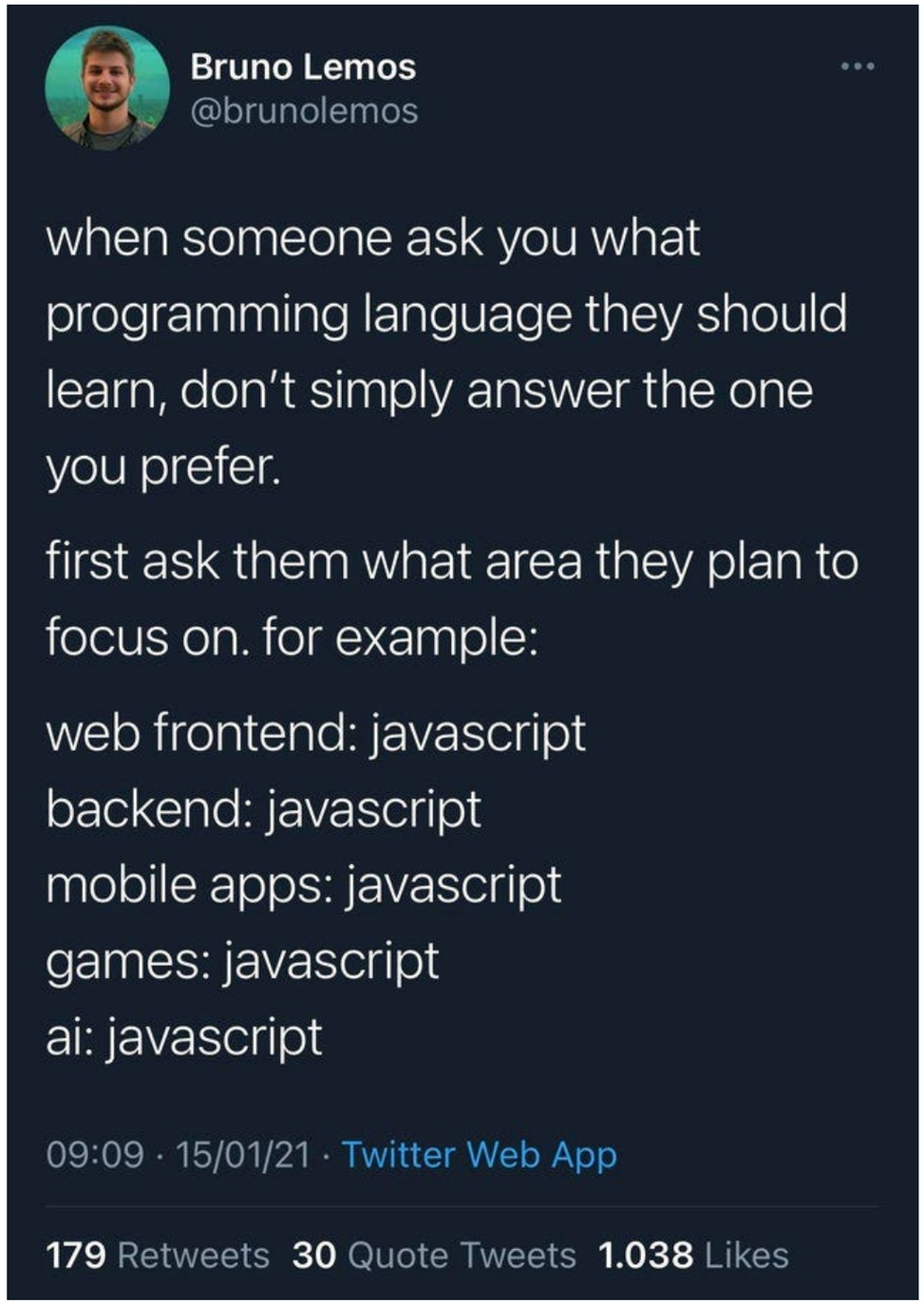
Usi comuni

1. Manipolazione del DOM per accedere e modificare elementi della pagina

2. Aggiornamento in tempo reale di contenuti, pop-up di finestre modali, transizioni animate

3. Gestione di eventi utente come click, scroll o input

4. Richieste asincrone (AJAX) per comunicare con il server, reperire e presentare informazioni senza ricaricare la pagina



Differenze con altri linguaggi

	JavaScript	Java	C++
Esecuzione	Interpretato	Compilato in bytecode, eseguito su JVM	Compilato in codice macchina
Tipizzazione	Debole, dinamica	Forte, statica	Forte, statica
Gestione memoria	Garbage collection	Garbage collection	Manuale
Oggetti	Basati su prototipi	Basati su classi	Basati su classi
Concorrenza	Event-driven, single-threaded	Supporta multi-threading	Supporta multi-threading
Uso principale	Interattività web, frontend/backend	Applicazioni enterprise, app Android	Programmazione di sistema, giochi

Esempio



Accendi la luce

HTML

```
<img id="luce-img" src="immagini/luce-off.png" alt="luce-off">
<button id="bott-luce" onclick="accendiLuce()">Accendi la luce</button>
```

JS

```
function accendiLuce() {
    document.getElementById("luce-img").src = "immagini/luce-on.png";
}
```

Esempio



HTML

```
<img id="luce-img" src="immagini/luce-off.png" alt="luce-off">
<button id="bott-luce" onclick="accendiLuce()">Accendi la luce</button>
```

JS

```
function accendiLuce() {
    document.getElementById("luce-img").src = "immagini/luce-on.png";
}
```

Dove inserire il js?

Posizione del JavaScript	Vantaggi	Svantaggi
Nel tag <head>	• Esecuzione rapida, prima del contenuto • Utile per script di inizializzazione o librerie	• Blocca il rendering della pagina • Rallenta il caricamento
In mezzo al tag <body>	• Utile quando lo script dipende da una parte specifica del contenuto	• Può causare blocchi nella parte successiva del contenuto • Richiede maggiore attenzione alla gestione del flusso di esecuzione, per evitare problemi di timing.
Fine del tag <body>	• Evita il blocco del rendering • Il DOM è già caricato, quindi lo script può interagire facilmente con la pagina	• Non utile per inizializzazioni rapide o per script da eseguire all'inizio • Ritardi nei contenuti interattivi
File esterno	• Separazione del codice e migliore mantenibilità • Caching del browser • Riutilizzo dello script su più pagine	• Richiesta HTTP aggiuntiva per ogni file esterno • Necessario usare async o defer per evitare blocchi

Dove inserire il js?

Posizione del JavaScript	Vantaggi	Svantaggi
Nel tag <head>	• Esecuzione rapida, prima del contenuto • Utile per script di inizializzazione o librerie	• Blocca il rendering della pagina • Rallenta il caricamento
In mezzo al tag <body>	• Utile quando lo script dipende da una parte specifica del contenuto	• Può causare blocchi nella parte successiva del contenuto • Richiede maggiore attenzione alla gestione del flusso di esecuzione, per evitare problemi di timing.
Fine del tag <body>	• Evita il blocco del rendering • Il DOM è già caricato, quindi lo script può interagire facilmente con la pagina	• Non utile per inizializzazioni rapide o per script da eseguire all'inizio • Ritardi nei contenuti interattivi
File esterno	• Separazione del codice e migliore mantenibilità • Caching del browser • Riutilizzo dello script su più pagine	• Richiesta HTTP aggiuntiva per ogni file esterno • Necessario usare async o defer per evitare blocchi

Esempio



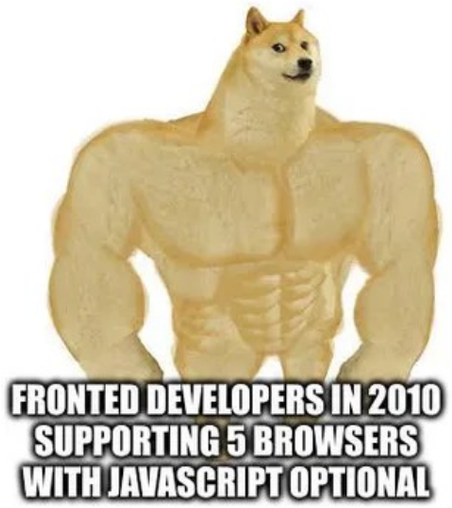
Accendi la luce

HTML

```
<head>
    <title>Esercizio</title>
    <meta charset="utf-8">
    <link rel="stylesheet" href="stile/stile.css">
    <script src="script/luce.js"></script>
</head>
```

Elementi principali

- Variabili
- Funzioni
- Manipolazione del DOM
- Operatori
- Strutture di controllo (if, else, while, for, switch)
- Array e Oggetti
- Eventi





Dichiarazione di variabili

1. **var**: ambito di visibilità globale o di funzione (più vecchia, sconsigliata)

2. **let**: ambito di blocco, ad esempio all'interno di un ciclo for (più moderna e sicura rispetto a var)

3. **const**: non possono essere riassegnate dopo la dichiarazione

JS

```
var y = 20;
let x = 10;
const pi = 3.14159;
```

Funzioni

- Eseguono blocchi di codice ed possono ritornare un risultato
- Tipi di funzione:
  - funzioni dichiarate

JS
```
function somma(a, b) {
    return a + b;
}
```

  - funzioni anonime

JS
```
let somma = function(a, b) {
    return a + b;
};
```

  - arrow function

JS
```
let somma = (a, b) => a + b;
```

Esercizio

- Al click di un bottone eseguire la somma di due numeri e mostrarla a schermo

HTML

```
<button onclick="somma(2,5)">Somma</button>
```

- Con **console.log**

JS

```
function somma(a, b) {
    let s = a+b;
    console.log(s);


}
```

Dove compare s?

Esercizio

- Al click di un bottone eseguire la somma di due numeri e mostrarla a schermo

HTML

```
<button onclick="somma(2,5)">Somma</button>
```

- Con **console.log**
- Con **window.alert**

JS

```
function somma(a, b) {
    let s = a+b;
    console.log(s);
    window.alert(s)

}
```

Dove compare s?

Esercizio

- Al click di un bottone eseguire la somma di due numeri e mostrarla a schermo

HTML

```
<button onclick="somma(2,5)">Somma</button>
<p id="ris"></p>
```

- Con **console.log**
- Con **window.alert**
- Con **innerHTML**

JS

```
function somma(a, b) {
    let s = a+b;
    console.log(s);
    window.alert(s)
    document.getElementById("ris").innerHTML = s;
}
```

Dove compare s?

Selezione degli elementi

- **document.getElementById("id")**: ritorna l'elemento del DOM con un determinato id
- **document.getElementsByTagName("tag")**: ritorna tutti gli elementi con un determinato tag sotto forma di HTMLCollection
- **document.querySelector("selettore")**: seleziona il primo elemento che corrisponde al selettore CSS specificato (id, classe o tag)
- **document.querySelectorAll("selettore")**: ritorna tutti gli elementi che corrispondono al selettore sotto forma di NodeList

NodeList e HTMLCollection

- **HTMLCollection**: raccolta di soli nodi di tipo **Element** (tag contenitori HTML)
- Le HTMLCollection generate con **getElementsByTagName** sono **dinamiche**
- Non supportano .forEach() e devono essere scorse in un ciclo for

- **NodeList**: elenco di nodi DOM che può contenere nodi Element, testi e commenti
- Le NodeList create con **querySelectorAll** sono **statiche**: non si aggiornano automaticamente se il DOM cambia
- Possono essere scorse con .forEach()

Esempio

JS

```
function provaSelezione() {
    const parByTagName = document.getElementsByTagName("p");
    const parQuerySelector = document.querySelectorAll("p");

    console.log("Prima - getElementsByTagName:", parByTagName);
    console.log("Prima - querySelectorAll:", parQuerySelector);

    const newP = document.createElement("p");
    newP.textContent = "Ciao, sono un altro paragrafo!";
    document.body.appendChild(newP);

    console.log("Dopo - getElementsByTagName:", parByTagName);
    console.log("Dopo - querySelectorAll:", parQuerySelector);
}
```

Genitori, figli e fratelli

Accesso	Nodi Element	Tutti i nodi
Figli	children	childNodes
Primo figlio	firstElementChild	firstChild
Ultimo figlio	lastElementChild	lastChild
Fratello successivo	nextElementSibling	nextSibling
Fratello precedente	previousElementSibling	previousSibling
Genitore	parentElement	parentNode

Creare elementi HTML

- **document.createElement()** con a parametro una stringa che rappresenta il nome del tag HTML

- **innerHTML** è una proprietà degli elementi HTML

- **elemento.innerHTML** ritorna il contenuto di **elemento**

- **elemento.innerHTML = “ Ciao! ”** sostituisce il contenuto di **elemento** con ** Ciao! **

Creare elementi HTML

- **document.createElement()** con a parametro una stringa che rappresenta il nome del tag HTML

- **innerHTML** è una proprietà degli elementi HTML

- **elemento.innerHTML** ritorna il contenuto di **elemento**

- **elemento.innerHTML = “ Ciao! ”** sostituisce il contenuto di **elemento** con ** Ciao! **

innerHTML comporta rischi di attacchi di cross-site scripting (XSS) quando si inietta HTML malevolo

Modifica delle Proprietà

- **textContent**: consente di ottenere o impostare il testo all'interno di un elemento HTML. Non interpreta eventuali tag HTML

JS

```
const elemento = document.getElementById("testoA");
console.log("Testo prima:", elemento.textContent);
elemento.textContent = "La bella addormentata";
console.log("Testo dopo:", elemento.textContent);
```

Modifica delle Proprietà

- **textContent**: consente di ottenere o impostare il testo all'interno di un elemento HTML. Non interpreta eventuali tag HTML

- **className**: consente di ottenere o impostare il valore dell'attributo class di un elemento HTML. Sovrascrive le classi esistenti

JS

```js
const elemento = document.getElementById("bello1");
elemento.className = "bott-bello-bis";
```

Modifica delle Proprietà

- **textContent**: consente di ottenere o impostare il testo all'interno di un elemento HTML. Non interpreta eventuali tag HTML

- **className**: consente di ottenere o impostare il valore dell'attributo class di un elemento HTML. Sovrascrive le classi esistenti

- **classList**: è un'interfaccia per manipolare le classi di un elemento HTML. Metodi: add(), remove(), toggle(), contains()

JS

```js
const elemento = document.getElementById("bello2");
elemento.classList.toggle("bott-bello-bis");
```

Modifica delle Proprietà

- **textContent**: consente di ottenere o impostare il testo all'interno di un elemento HTML. Non interpreta eventuali tag HTML

- **className**: consente di ottenere o impostare il valore dell'attributo class di un elemento HTML. Sovrascrive le classi esistenti

- **classList**: è un'interfaccia per manipolare le classi di un elemento HTML. Metodi: add(), remove(), toggle(), contains()

- **setAttribute**: imposta il valore di un attributo specificato di un elemento

JS

```js
const img = document.getElementById("paese");
img.setAttribute("src", "immagini/imm2.jpg");
```

Modifica delle Proprietà

- **textContent**: consente di ottenere o impostare il testo all'interno di un elemento HTML. Non interpreta eventuali tag HTML

- **className**: consente di ottenere o impostare il valore dell'attributo class di un elemento HTML. Sovrascrive le classi esistenti

- **classList**: è un'interfaccia per manipolare le classi di un elemento HTML. Metodi: add(), remove(), toggle(), contains()

- **setAttribute**: imposta il valore di un attributo specificato di un elemento

- **getAttribute**: ottiene il valore di un attributo

JS

```js
const img = document.getElementById("paese");
img.getAttribute("src");
```

Proprietà .value

- Utilizzata per leggere o impostare il valore di un input
- **document.getElementById(“nome”).value**: restituisce il contenuto dell’input con id="input"
- **document.getElementById(“nome”).value = “Marco”**: inserisce il testo specificato come valore



Modifica degli Stili

- Utile per effetti dinamici
- Meno performante rispetto alla gestione tramite CSS
- **elemento.style.property = “valore”**: cambia lo stile in linea di un elemento

JS

```
let elemento = document.getElementById("mio-div");
elemento.style.backgroundColor = "#c00083";
elemento.style.fontSize = "2em";
```

- **setAttribute** e **classList:** per manipolare style e le classi

Inserimento nel DOM

- **el.appendChild(node)**: inserisce **node** come ultimo figlio di **el**
- **el.prepend(node)**: inserisce **node** come primo figlio di **el**
- **el.insertBefore(node, refNode)**: inserisce **newNode** prima di **refNode**
- **el.replaceChild(newNode, oldNode)**: sostituisce **oldNode** con **newNode**

Esercizio

HTML

```
<body>
    <h1>Supereroi</h1>
    <p id="el">I supereroi sono personaggi con poteri straordinari.</p>
    <h2>Supereroi Famosi</h2>
    <p id="par">Superman, Batman, Spider-Man: eroi che ispirano milioni di persone.</p>
</body>
```

JS

```
let el = document.getElementById("el");
let nodo = document.createElement(“p”);
nodo.textContent = "Molti supereroi volano";
```

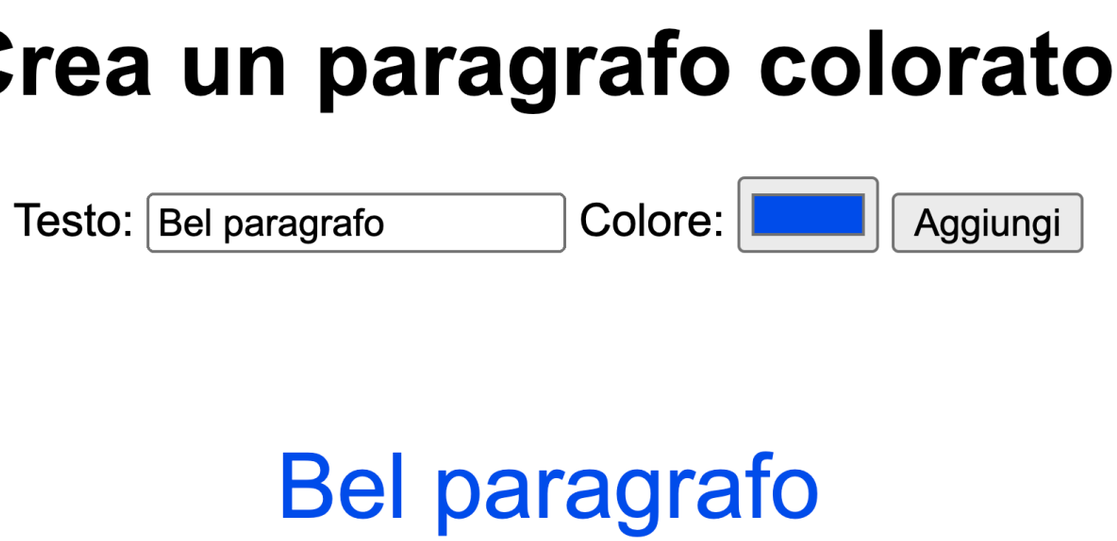
- Dove viene inserito nodo se uso **document.body.**
- **appendChild(nodo)**?
- **insertBefore(nodo, par)**?
- **prepend(nodo)**?
- **replaceChild(nodo, el)**?

Rimozione dal DOM

- **el.innerHTML = ""**: elimina tutti gli elementi figli di **el**
- **el.removeChild(node)**: elimina **node** dal genitore **el**
- **el.remove()**: elimina **el** direttamente

Esercizio

- Creare una pagina con una form che chiede all'utente di inserire un testo e un colore
- A click del bottone di submit, viene aggiunto alla pagina un paragrafo con il testo e il colore specificati



Operatori

- Simboli speciali utilizzati per eseguire operazioni su valori e variabili
- Operatori **aritmetici**: calcoli matematici di base (**+**, **-**, ***, **/**, **%**)
- Operatori di **assegnazione**: assegnano valori a variabili (**=**, **+=**, **-=**)
- Operatori di **confronto**: confrontano due valori e restituiscono un valore booleano (**==**, **!=**, **===**, **<**, **>**)
- Operatori **logici**: utilizzati per espressioni condizionali (**&&**, **||**, **!**)
- Operatori **bitwise**: operano sui singoli bit di un numero (**&**, **|**, **^**, **~**)

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false` ?
   - `0 === false` ?

2. **Conversioni implicite**
   - `5 + "5"` ?
   - `"5" - 5` ?

3. **+ unario**
   - `+"5"` ?
   - `+"hello"` ?

4. **Confronti**
   - `null == undefined` ?
   - `null === undefined` ?
   - `NaN == NaN` ?
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false` ?

2. **Conversioni implicite**
   - `5 + "5"` ?
   - `"5" - 5` ?

3. **+ unario**
   - `+"5"` ?
   - `+"hello"` ?

4. **Confronti**
   - `null == undefined` ?
   - `null === undefined` ?
   - `NaN == NaN` ?
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"` ?
   - `"5" - 5` ?

3. **+ unario**
   - `+"5"` ?
   - `+"hello"` ?

4. **Confronti**
   - `null == undefined` ?
   - `null === undefined` ?
   - `NaN == NaN` ?
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5` ?

3. **+ unario**
   - `+"5"` ?
   - `+"hello"` ?

4. **Confronti**
   - `null == undefined` ?
   - `null === undefined` ?
   - `NaN == NaN` ?
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"` ?
   - `+"hello"` ?

4. **Confronti**
   - `null == undefined` ?
   - `null === undefined` ?
   - `NaN == NaN` ?
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"  // 5`
   - `+"hello"` ?

4. **Confronti**
   - `null == undefined` ?
   - `null === undefined` ?
   - `NaN == NaN` ?
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"      // 5`
   - `+"hello"  // NaN`

4. **Confronti**
   - `null == undefined` ?
   - `null === undefined` ?
   - `NaN == NaN` ?
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"      // 5`
   - `+"hello"  // NaN`

4. **Confronti**
   - `null == undefined  // true`
   - `null === undefined` ?
   - `NaN == NaN` ?
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"      // 5`
   - `+"hello"  // NaN`

4. **Confronti**
   - `null == undefined  // true`
   - `null === undefined // false`
   - `NaN == NaN` **?**
   - `NaN === NaN` **?**
   - `isNaN(NaN)` **?**
   - `[] == false` **?**
   - `[] == ""` **?**

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"      // 5`
   - `+"hello"  // NaN`

4. **Confronti**
   - `null == undefined  // true`
   - `null === undefined // false`
   - `NaN == NaN    // false`
   - `NaN === NaN` ?
   - `isNaN(NaN)` ?
   - `[] == false` ?
   - `[] == ""` ?

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"      // 5`
   - `+"hello"  // NaN`

4. **Confronti**
   - `null == undefined  // true`
   - `null === undefined // false`
   - `NaN == NaN    // false`
   - `NaN === NaN   // false`
   - `isNaN(NaN)` **?**
   - `[] == false` **?**
   - `[] == ""` **?**

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"      // 5`
   - `+"hello"  // NaN`

4. **Confronti**
   - `null == undefined  // true`
   - `null === undefined // false`
   - `NaN == NaN    // false`
   - `NaN === NaN   // false`
   - `isNaN(NaN)    // true`
   - `[] == false` **?**
   - `[] == ""` **?**

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"      // 5`
   - `+"hello"  // NaN`

4. **Confronti**
   - `null == undefined  // true`
   - `null === undefined // false`
   - `NaN == NaN    // false`
   - `NaN === NaN   // false`
   - `isNaN(NaN)    // true`
   - `[] == false   // true`
   - `[] == ""` **?**

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

2. **Conversioni implicite**
   - `5 + "5"  // "55"`
   - `"5" - 5  // 0`

3. **+ unario**
   - `+"5"      // 5`
   - `+"hello"  // NaN`

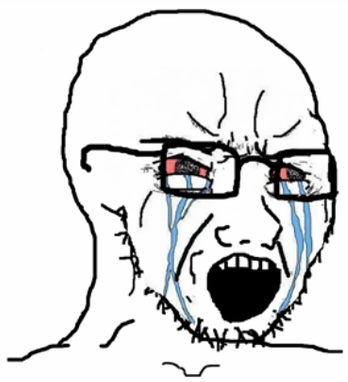
4. **Confronti**
   - `null == undefined  // true`
   - `null === undefined // false`
   - `NaN == NaN    // false`
   - `NaN === NaN   // false`
   - `isNaN(NaN)    // true`
   - `[] == false   // true`
   - `[] == ""      // true`

Comportamenti inaspettati

1. **== vs ===**
   - `0 == false  // true`
   - `0 === false // false`

“Man, your programming language really sucks”

PHP devs



“Nooooooo 90% of web pages are written in PHP it’s a good language and it will remain useful for a long time”

JavaScript devs



“Yes”

4. **Confronti**
   - `null == undefined  // true`
   - `null === undefined // false`
   - `NaN == NaN    // false`
   - `NaN === NaN   // false`
   - `isNaN(NaN)    // true`
   - `[] == false   // true`
   - `[] == ""      // true`